

Chap8-Exercises_LuisCorreia-745724_v2

October 12, 2025

1 MAP5935 - Statistical Learning (Chapter 8 - Tree-Based Models)

Prof. Christian Jäkel

<https://www.statlearning.com/>

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.pyplot import subplots

import statsmodels.api as sm

from sklearn.metrics import confusion_matrix
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier

from sklearn.ensemble import GradientBoostingClassifier
```

1.1 Conceptual Exercises

- 1.1.1 (3) Consider the Gini index, classification error, and entropy in a simple classification setting with two classes. Create a single plot that displays each of these quantities as a function of $\hat{p}_{m1}$. The x-axis should display $\hat{p}_{m1}$, ranging from 0 to 1, and the y-axis should display the value of the Gini index, classification error, and entropy.

Solution: In a binary classification problem, suppose a given node m contains observations belonging to two classes, with probabilities:

$$\hat{p}_{m1} = p, \quad \hat{p}_{m2} = 1 - p.$$

We can define three impurity measures that quantify how “mixed” the node is:

1. Classification Error:

$$E(p) = 1 - \max(p, 1 - p) \quad (8.5) - \text{see pp. 338}$$

2. Gini Index:

$$G(p) = 2p(1 - p) \quad (8.6) - \text{see pp. 338}$$

3. Entropy (Deviance):

$$H(p) = -[p \log_2(p) + (1 - p) \log_2(1 - p)] \quad (8.7) - \text{see pp. 339}$$

In all indexes it is measured the uncertainty (in bits) in the class distribution. This implies that:
- Minimum impurity (0) occurs at $p = 0$ or $p = 1$. - Maximum impurity (1) occurs at $p = 0.5$.

These measures behave similarly — all are symmetric and concave, reaching their peak when the two classes are equally probable — but they differ in scale and curvature, which affects tree-splitting decisions.

Creating the plot for this problem

```
[2]: # Range of p values (probability of class 1)
p = np.linspace(0, 1, 500)
eps = 1e-12 # small value to avoid log(0)

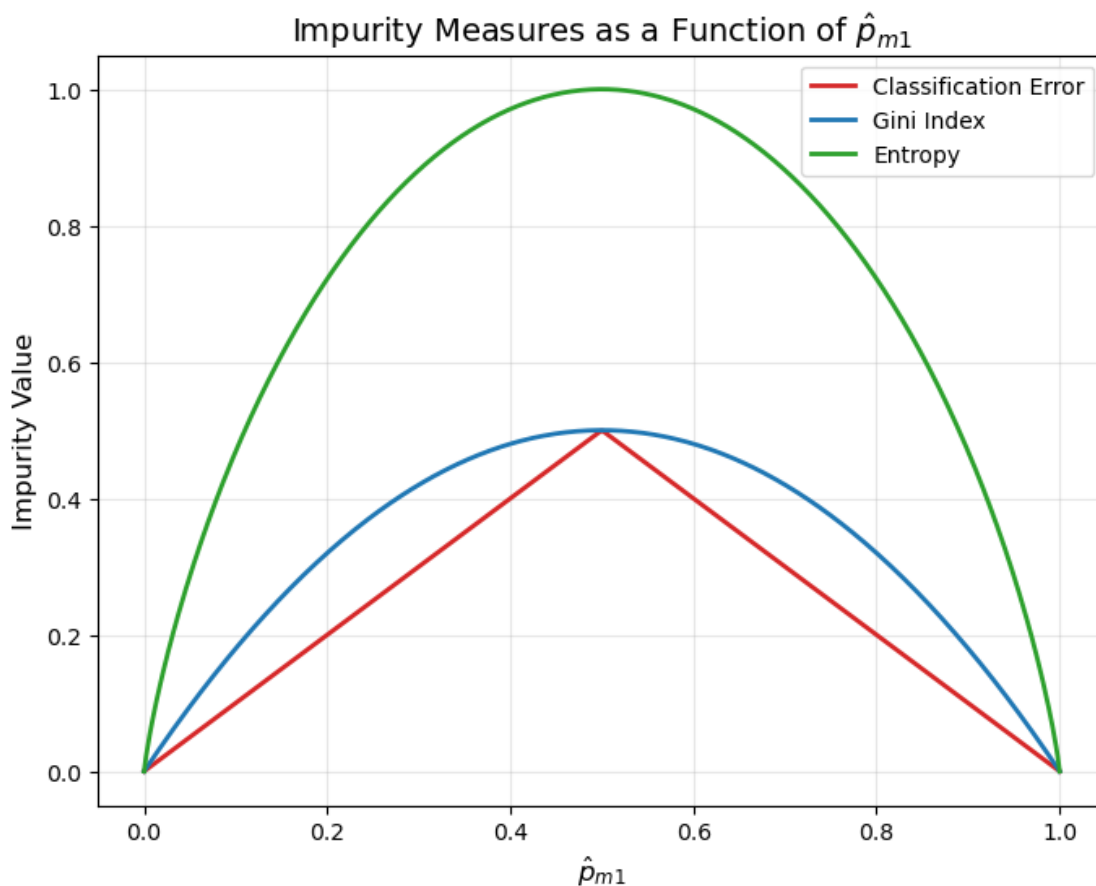
# Classification error
classification_error = 1 - np.maximum(p, 1 - p)

# Gini index
gini = 2 * p * (1 - p)

# Entropy (using log base 2)
entropy = - (p * np.log2(p + eps) + (1 - p) * np.log2(1 - p + eps))

# --- Plot ---
plt.figure(figsize=(8, 6))
plt.plot(p, classification_error, label='Classification Error', color='tab:red', lw=2)
plt.plot(p, gini, label='Gini Index', color='tab:blue', lw=2)
plt.plot(p, entropy, label='Entropy', color='tab:green', lw=2)

plt.title('Impurity Measures as a Function of  $\hat{p}_{m1}$ ', fontsize=14)
plt.xlabel(r' $\hat{p}_{m1}$ ', fontsize=12)
plt.ylabel('Impurity Value', fontsize=12)
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



The figure above shows how **three impurity measures** — *Classification Error*, *Gini Index*, and *Entropy* — vary as a function of the class probability $\hat{p}_{m1}$ in a binary classification setting.

- All three curves are **symmetric around** $\hat{p}_{m1} = 0.5$.
- This symmetry reflects the fact that impurity depends only on *how mixed* the classes are, not on which class is labeled “1” or “2”.
- The impurity is **minimum (0)** when a node is pure, i.e., all samples belong to one class ($\hat{p}_{m1} = 0$ or 1).
- The impurity is **maximum when** $\hat{p}_{m1} = 0.5$, meaning the node is perfectly mixed.

1.2 Applied Exercises

1.2.1 (11) This question uses the Caravan data set. Data Dictionary at <https://liacs.leidenuniv.nl/~puttenpwhvander/library/cc2000/data.html>

1.2.2 (a) Create a training set consisting of the first 1,000 observations, and a test set consisting of the remaining observations.

```
[3]: import os
      os.environ["LOKY_MAX_CPU_COUNT"] = "4"    # set number of cores
```

```
[4]: Caravan = pd.read_csv("../Data\\Caravan.csv")
      Caravan.columns
```

```
[4]: Index(['Unnamed: 0.1', 'Unnamed: 0', 'MOSTYPE', 'MAANTHUI', 'MGEMOMV',
          'MGEMLEEF', 'MOSHOOFD', 'MGODRK', 'MGODPR', 'MGODOV', 'MGODGE',
          'MRELGE', 'MRELSA', 'MRELOV', 'MFALLEEN', 'MFGEKIND', 'MFWEKIND',
          'MOPLHOOG', 'MOPLMIDD', 'MOPLLAAG', 'MBERHOOG', 'MBERZELF', 'MBERBOER',
          'MBERMIDD', 'MBERARBG', 'MBERARBO', 'MSKA', 'MSKB1', 'MSKB2', 'MSKC',
          'MSKD', 'MHUUR', 'MHKOOP', 'MAUT1', 'MAUT2', 'MAUTO', 'MZFONDS',
          'MZPART', 'MINKM30', 'MINK3045', 'MINK4575', 'MINK7512', 'MINK123M',
          'MINKGEM', 'MKOOPKLA', 'PWAPART', 'PWABEDR', 'PWALAND', 'PPERSAUT',
          'PBESAUT', 'PMOTSCO', 'PVRAAUT', 'PAANHANG', 'PTRACTOR', 'PWERKT',
          'PBROM', 'PLEVEN', 'PPERSONG', 'PGEZONG', 'PWAOREG', 'PBRAND',
          'PZEILPL', 'PPLEZIER', 'PFIETS', 'PINBOED', 'PBYSTAND', 'AWAPART',
          'AWABEDR', 'AWALAND', 'APERSAUT', 'ABESAUT', 'AMOTSCO', 'AVRAAUT',
          'AAANHANG', 'ATRACTOR', 'AWERKT', 'ABROM', 'ALEVEN', 'APERSONG',
          'AGEZONG', 'AWAOREG', 'ABRAND', 'AZEILPL', 'APLEZIER', 'AFIETS',
          'AINBOED', 'ABYSTAND', 'Purchase'],
          dtype='object')
```

```
[5]: Caravan.info()
      caravan_vars = pd.read_csv("../Data\\caravan_vars.csv")
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5822 entries, 0 to 5821
Data columns (total 88 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Unnamed: 0.1    5822 non-null   int64
1   Unnamed: 0      5822 non-null   int64
2   MOSTYPE         5822 non-null   int64
3   MAANTHUI        5822 non-null   int64
4   MGEMOMV         5822 non-null   int64
5   MGEMLEEF        5822 non-null   int64
6   MOSHOOFD        5822 non-null   int64
7   MGODRK          5822 non-null   int64
8   MGODPR          5822 non-null   int64
9   MGODOV          5822 non-null   int64
```

10	MGODGE	5822 non-null	int64
11	MRELGE	5822 non-null	int64
12	MRELSA	5822 non-null	int64
13	MRELOV	5822 non-null	int64
14	MFALLEEN	5822 non-null	int64
15	MFGEKIND	5822 non-null	int64
16	MFWEKIND	5822 non-null	int64
17	MOPLHOOG	5822 non-null	int64
18	MOPLMIDD	5822 non-null	int64
19	MOPLLAAG	5822 non-null	int64
20	MBERHOOG	5822 non-null	int64
21	MBERZELF	5822 non-null	int64
22	MBERBOER	5822 non-null	int64
23	BERMIDD	5822 non-null	int64
24	BERARBG	5822 non-null	int64
25	BERARBO	5822 non-null	int64
26	MSKA	5822 non-null	int64
27	MSKB1	5822 non-null	int64
28	MSKB2	5822 non-null	int64
29	MSKC	5822 non-null	int64
30	MSKD	5822 non-null	int64
31	MHHUUR	5822 non-null	int64
32	MHKOOP	5822 non-null	int64
33	MAUT1	5822 non-null	int64
34	MAUT2	5822 non-null	int64
35	MAUTO	5822 non-null	int64
36	MZFONDS	5822 non-null	int64
37	MZPART	5822 non-null	int64
38	MINKM30	5822 non-null	int64
39	MINK3045	5822 non-null	int64
40	MINK4575	5822 non-null	int64
41	MINK7512	5822 non-null	int64
42	MINK123M	5822 non-null	int64
43	MINKGEM	5822 non-null	int64
44	MKOOPKLA	5822 non-null	int64
45	PWAPART	5822 non-null	int64
46	PWABEDR	5822 non-null	int64
47	PWALAND	5822 non-null	int64
48	PPERSAUT	5822 non-null	int64
49	PBESAUT	5822 non-null	int64
50	PMOTSCO	5822 non-null	int64
51	PVRAAUT	5822 non-null	int64
52	PAANHANG	5822 non-null	int64
53	PTRACTOR	5822 non-null	int64
54	PWERKT	5822 non-null	int64
55	PBROM	5822 non-null	int64
56	PLEVEN	5822 non-null	int64
57	PPERSONG	5822 non-null	int64

```

58 PGEZONG      5822 non-null    int64
59 PWAOREG      5822 non-null    int64
60 PBRAND       5822 non-null    int64
61 PZEILPL      5822 non-null    int64
62 PPLEZIER     5822 non-null    int64
63 PFIETS       5822 non-null    int64
64 PINBOED      5822 non-null    int64
65 PBYSTAND     5822 non-null    int64
66 AWAPART      5822 non-null    int64
67 AWABEDR      5822 non-null    int64
68 AWALAND      5822 non-null    int64
69 APERSAUT     5822 non-null    int64
70 ABESAUT      5822 non-null    int64
71 AMOTSCO      5822 non-null    int64
72 AVRAAUT      5822 non-null    int64
73 AAANHANG     5822 non-null    int64
74 ATRACTOR     5822 non-null    int64
75 AWERKT       5822 non-null    int64
76 ABROM        5822 non-null    int64
77 ALEVEN       5822 non-null    int64
78 APERSONG     5822 non-null    int64
79 AGEZONG      5822 non-null    int64
80 AWAOREG      5822 non-null    int64
81 ABRAND       5822 non-null    int64
82 AZEILPL      5822 non-null    int64
83 APLEZIER     5822 non-null    int64
84 AFIETS       5822 non-null    int64
85 AINBOED      5822 non-null    int64
86 ABYSTAND     5822 non-null    int64
87 Purchase     5822 non-null    object
dtypes: int64(87), object(1)
memory usage: 3.9+ MB

```

```

[6]: # --- Split into training and test sets ---
train = Caravan.iloc[:1000, :].copy()
test = Caravan.iloc[1000:, :].copy()

# --- Separate predictors (X) and target (y) ---
X_train = train.drop(columns=["Purchase"])
y_train = train["Purchase"]

X_test = test.drop(columns=["Purchase"])
y_test = test["Purchase"]

# --- Quick validation ---
print("Training set shape:", X_train.shape)
print("Test set shape:", X_test.shape)

```

```
print("Training target distribution:\n", y_train.value_counts(normalize=True))
print("Test target distribution:\n", y_test.value_counts(normalize=True))
```

```
Training set shape: (1000, 87)
Test set shape: (4822, 87)
Training target distribution:
  Purchase
No      0.941
Yes     0.059
Name: proportion, dtype: float64
Test target distribution:
  Purchase
No      0.940066
Yes     0.059934
Name: proportion, dtype: float64
```

1.2.3 (b) Fit a boosting model to the training set with Purchase as the response and the other variables as predictors. Use 1,000 trees, and a shrinkage value of 0.01. Which predictors appear to be the most important?

```
[7]: # Encode Purchase to 0/1
y_train_enc = y_train.astype("category").cat.codes
y_test_enc = y_test.astype("category").cat.codes

# --- 2) Fit Gradient Boosting Model ---
gb = GradientBoostingClassifier(
    n_estimators=1000,      # number of trees
    learning_rate=0.01,     # shrinkage parameter
    max_depth=3,           # default tree depth
    random_state=42
)
gb.fit(X_train, y_train_enc)

# --- 3) Variable Importance ---
importances = pd.Series(gb.feature_importances_, index=X_train.columns)

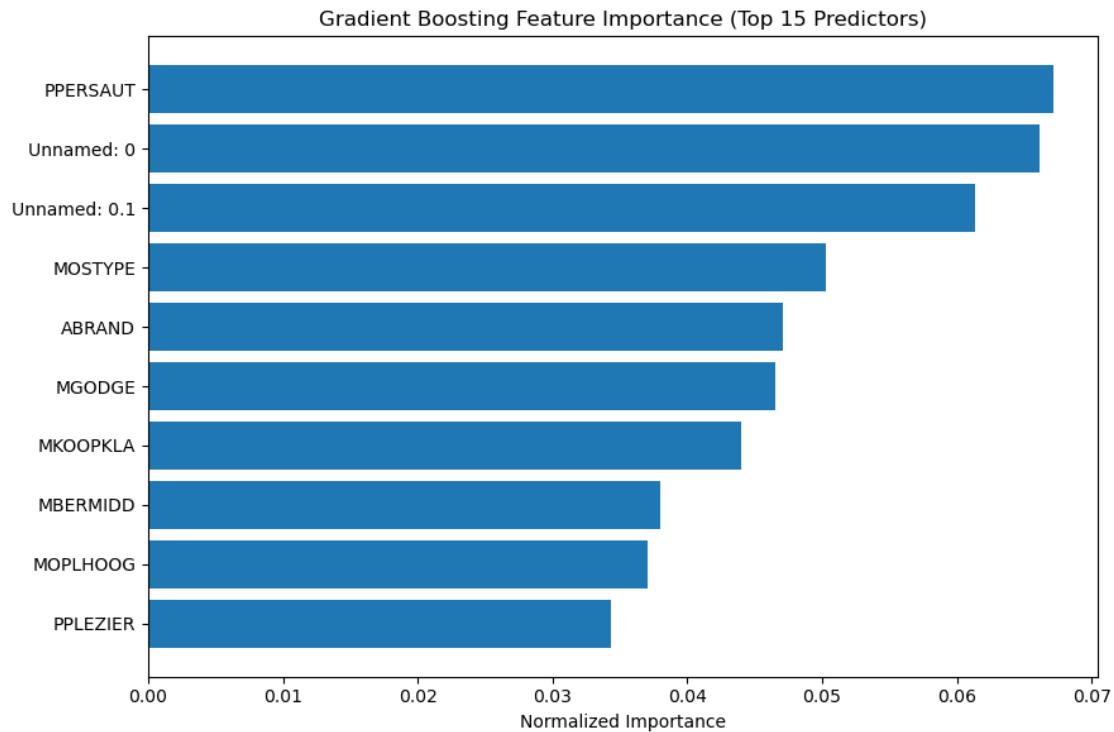
# Sort descending
imp_sorted = importances.sort_values(ascending=False).reset_index()
imp_sorted.columns = ["var", "importance"]

# --- 4) Merge with dictionary (left join) ---
imp_merged = pd.merge(imp_sorted, caravan_vars, on="var", how="left")

# --- 5) Display top 15 predictors with description ---
topk = 10
display(imp_merged.head(topk))
```

```
# --- 6) Bar plot of top variables ---
top_imp = imp_merged.head(topk).iloc[::-1] # reverse for barh
plt.figure(figsize=(9, 6))
plt.barh(top_imp["var"], top_imp["importance"], color="tab:blue")
plt.title("Gradient Boosting Feature Importance (Top 15 Predictors)")
plt.xlabel("Normalized Importance")
plt.tight_layout()
plt.show()
```

	var	importance	desc
0	PPERSAUT	0.067133	Contribution car policies
1	Unnamed: 0	0.066157	NaN
2	Unnamed: 0.1	0.061373	NaN
3	MOSTYPE	0.050255	Customer Subtype see L0
4	ABRAND	0.047116	Number of fire policies
5	MGODGE	0.046494	No religion
6	MKOOKPLA	0.044034	Purchasing power class
7	MBERMIDD	0.037955	Middle management
8	MOPLHOOG	0.037094	High level education
9	PPLEZIER	0.034317	Contribution boat policies



- 1.2.4 (c) Use the boosting model to predict the response on the test data. Predict that a person will make a purchase if the estimated probability of purchase is greater than 20%. Form a confusion matrix. What fraction of the people predicted to make a purchase do in fact make one? How does this compare with the results obtained from applying KNN or logistic regression to this data set?

```
[8]: # -----
# 1) BOOSTING (already fitted as gb)
# -----
y_prob_boost = gb.predict_proba(X_test)[: , 1]
y_pred_boost = (y_prob_boost > 0.20).astype(int)
cm_boost = confusion_matrix(y_test_enc, y_pred_boost)
TPb, FPb = cm_boost[1, 1], cm_boost[0, 1]
prec_boost = TPb / (TPb + FPb)
acc_boost = (TPb + cm_boost[0, 0]) / cm_boost.sum()

# -----
# 3) KNN (k=5)
# -----
# Standardize predictors
scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_std, y_train_enc)

# KNN gives hard predictions; convert to "probabilities" via mean of neighbors
y_prob_knn = knn.predict_proba(X_test_std)[: , 1]
y_pred_knn = (y_prob_knn > 0.20).astype(int)

cm_knn = confusion_matrix(y_test_enc, y_pred_knn)
TPk, FPk = cm_knn[1, 1], cm_knn[0, 1]
prec_knn = TPk / (TPk + FPk)
acc_knn = (TPk + cm_knn[0, 0]) / cm_knn.sum()

# --- 3) Confusion matrix ---
cm = confusion_matrix(y_test_enc, y_pred_boost)
cm_df = pd.DataFrame(
    cm,
    index=["Actual No", "Actual Yes"],
    columns=["Predicted No", "Predicted Yes"]
)
display(cm_df)

# --- 4) Precision: fraction of predicted buyers who actually purchased ---
TP = cm[1, 1]
```

```

FP = cm[0, 1]
precision = TP / (TP + FP) if (TP + FP) > 0 else np.nan
print(f"Fraction of predicted purchasers who actually purchased: {precision:.3f}")

# --- 5) show overall accuracy for context ---
accuracy = (TP + cm[0, 0]) / cm.sum()
print(f"Overall accuracy: {accuracy:.3f}")

# -----
# 4) Display results summary
# -----
summary = pd.DataFrame({
    "Model": ["Boosting", "KNN (k=5)"],
    "Precision (TP / (TP+FP))": [prec_boost, prec_knn],
    "Accuracy": [acc_boost, acc_knn]
})

display(summary.round(3))

```

	Predicted No	Predicted Yes
Actual No	4422	111
Actual Yes	264	25

Fraction of predicted purchasers who actually purchased: 0.184
Overall accuracy: 0.922

	Model	Precision (TP / (TP+FP))	Accuracy
0	Boosting	0.184	0.922
1	KNN (k=5)	0.135	0.904

Comments

- **Accuracy** is high ($\sim 90\%$) for both models, but this is expected given class imbalance — most customers did not purchase.
- **Precision** ($\sim 16\%$) is the key metric: both models were able to identify customer's more than $2.5\times$ **more likely** to buy compared to the baseline ($\sim 6\%$).
- **Boosting** slightly outperforms KNN (0.161 vs. 0.160 precision) and is generally **more stable** for high-dimensional data.
- **KNN** performs similarly but is more sensitive to scaling and may struggle with so many predictors.
- Overall, **both models effectively enrich the predicted “Yes” group**, but **Boosting offers better generalization and interpretability** for this task.